

TEMA 7 – Unique Activity

Objetivo del tema

En este tema es conocer los pormenores del patrón de arquitectura Unique Activity, así como un uso elemental de las librerías Jetpack diseñadas para esta arquitectura.

7.1 Unique Activity

La arquitectura **Single Activity** (también llamada *Unique Activity*) consiste en diseñar una aplicación Android utilizando una única Activity como contenedor principal, mientras que toda la navegación y la interfaz se gestiona mediante Fragments.

La **forma clásica** de crear una App se basaba en esta lógica:

- Una Actividad es lo mismo que decir una ‘pantalla’
- Los Intent se usan para hacer la navegación entre diferentes ‘pantallas’.

¿Esta solución es errónea? No. De hecho, es una solución válida, aunque presenta varios problemas. Por mencionar uno, si tenemos tres o cuatro ‘pantallas’ diferentes en la App y pretendemos movernos por ellas libremente, la implementación se vuelve confusa. A demás, si tenemos varias Actividades en segundo plano, estamos consumiendo un montón de recursos en algo que está inactivo.

La **idea moderna** se centra en hacer que la Actividad sea solamente un marco en el que ir encajando las diferentes ‘piezas’ que conforman la App. Y por supuesto, cada una de esas ‘piezas’ es un Fragment.

- Una sola Activity, una sola ‘pantalla’
- Un Fragment es solamente una pieza de esa ‘pantalla’.
- Cada Fragment tiene su Ciclo de Vida propio.

Esto ofrece múltiples ventajas. Facilita la navegación dentro de tu App, la gestión de recursos y a demás podemos utilizar Jetpack para añadir toda una nueva serie de componentes a nuestra App.

7.2 Jetpack

Android Jetpack es un conjunto de librerías, componentes y herramientas oficiales de Google diseñadas para facilitar el desarrollo de aplicaciones Android modernas, seguras y mantenibles. Su objetivo es reducir el código repetitivo y promover buenas prácticas de arquitectura. Ofrece diferentes librerías, las principales son:

- **Navigation Component:** Permite gestionar la navegación entre Fragments.
- **ViewModel:** Almacenar y gestionar datos relacionados con la UI de forma consciente del ciclo de vida.
- **LiveData:** Permite observar cambios en los datos y actualizar la interfaz automáticamente.
- **Room:** Librería de persistencia para bases de datos SQLite de forma simplificada.
- **DataStore:** Almacenar datos simples de configuración (alternativa a **SharedPreferences**).
- **Paging:** Carga eficiente de grandes listas de datos
- **WorkManager:** Ejecuta tareas en segundo plano de forma fiable.

7.3 Navigation Component

Vamos a recuperar el ejemplo que hemos utilizado al explicar los Fragments: la App que lista una serie de mensajes de correo y muestra luego su detalle.

Paso 1 – Crea el proyecto: crea un proyecto nuevo, pero:

- En lugar de seleccionar *Empty Views Activity* escoge *Basic Views Activity*
- El valor de *Build Configuration Language* tiene que ser **Kotlin DSL**.

Paso 2 – Añadir Jetpack: vamos al Gradle y añadimos las librerías que necesitamos. Para ello, abrimos el fichero **build.gradle.kts** (module). Los ficheros de Gradle tienen que sincronizarse, no sirve con guardarlos. Verás que el propio IDE te avisa para que lo hagas cada vez que haces cambios. Android Studio necesita estos ficheros para compilar la App, por lo tanto, cualquier error debe de ser subsanado inmediatamente.

Escribe lo siguiente al final del bloque marcado como dependencias {...} y no te olvides de **Sincronizar**.

```
// Navigation Component
implementation("androidx.navigation:navigation-fragment-ktx:2.7.7")
implementation("androidx.navigation:navigation-ui-ktx:2.7.7")

// Fragment + Activity KTX
implementation("androidx.fragment:fragment-ktx:1.6.2")
implementation("androidx.activity:activity-ktx:1.8.2")

// ViewModel (recomendado en Jetpack)
implementation("androidx.lifecycle:lifecycle-viewmodel-ktx:2.7.0")
implementation("androidx.lifecycle:lifecycle-livedata-ktx:2.7.0")
```

(Opcional) Android Studio hace un seguimiento de las versiones de las librerías, y te permite con un simple clic obtener las últimas versiones. Dado que aún no hemos escrito una sola línea de código, es un buen momento para hacerlo. Veremos que nos deja algo así:

```
// Navigation Component
implementation("androidx.navigation:navigation-fragment-ktx:2.9.7")
implementation("androidx.navigation:navigation-ui-ktx:2.9.7")

// Fragment + Activity KTX
implementation("androidx.fragment:fragment-ktx:1.8.9")
implementation("androidx.activity:activity-ktx:1.13.0")

// ViewModel (recomendado en Jetpack)
implementation("androidx.lifecycle:lifecycle-viewmodel-ktx:2.10.0")
implementation("androidx.lifecycle:lifecycle-livedata-ktx:2.10.0")
```

(Opcional) Adicionalmente, podemos utilizar un catálogo de versiones, una herramienta que permite tener separadas las librerías y las versiones de estas en ficheros separados por claridad. Las versiones se almacenan en el **libs.versions.toml**, dejando en el **build.gradle.kts** (module) las referencias.

De nuevo, Android Studio permite ‘trasladar’ las versiones al **libs.versions.toml** con un solo clic.

build.gradle.kts (module) quedaría así:

```
// Navigation Component
implementation(libs.androidx.navigation.fragment.ktx)
implementation(libs.androidx.navigation.ui.ktx)

// Fragment + Activity KTX
implementation(libs.androidx.fragment.ktx)
implementation(libs.androidx.activity.ktx)

// ViewModel (recomendado en Jetpack)
implementation(libs.lifecycle.viewmodel.ktx)
implementation(libs.androidx.lifecycle.livedata.ktx)
```

Y el **libs.versions.toml** quedaría así:

```
[versions]
activityKtx = "1.13.0"
agp = "9.1.1"
coreKtx = "1.18.0"
fragmentKtx = "1.18.9"
junit = "4.13.2"
junitVersion = "1.3.0"
espressoCore = "3.7.0"
appcompat = "1.7.1"
lifecycleViewModelKtxVersion = "2.10.0"
material = "1.13.0"
activity = "1.13.0"
constraintlayout = "2.2.1"
navigationFragmentKtx = "2.9.7"

[libraries]
androidx-activity-ktx = { module = "androidx.activity:activity-ktx", version.ref = "activityKtx" }
androidx-core-ktx = { group = "androidx.core", name = "core-ktx", version.ref = "coreKtx" }
androidx-fragment-ktx = { module = "androidx.fragment:fragment-ktx", version.ref = "fragmentKtx" }
androidx-lifecycle-livedata-ktx = { module = "androidx.lifecycle:lifecycle-livedata-ktx", version.ref = "lifecycleViewModelKtxVersion" }
androidx-navigation-fragment-ktx = { module = "androidx.navigation:navigation-fragment-ktx", version.ref = "navigationFragmentKtx" }
androidx-navigation-ui-ktx = { module = "androidx.navigation:navigation-ui-ktx", version.ref = "navigationFragmentKtx" }
junit = { group = "junit", name = "junit", version.ref = "junit" }
androidx-junit = { group = "androidx.test.ext", name = "junit", version.ref = "junitVersion" }
androidx-espresso-core = { group = "androidx.test.espresso", name = "espresso-core", version.ref = "espressoCore" }
androidx-appcompat = { group = "androidx.appcompat", name = "appcompat", version.ref = "appcompat" }
lifecycle-viewmodel-ktx = { module = "androidx.lifecycle:lifecycle-viewmodel-ktx", version.ref = "lifecycleViewModelKtxVersion" }
material = { group = "com.google.android.material", name = "material", version.ref = "material" }
androidx-activity = { group = "androidx.activity", name = "activity", version.ref = "activity" }
androidx-constraintlayout = { group = "androidx.constraintlayout", name = "constraintlayout", version.ref = "constraintlayout" }

[plugins]
android-application = { id = "com.android.application", version.ref = "agp" }
```

Fíjate en cómo se mantienen las referencias entre ambos ficheros y las versiones.

Paso 3 – Activar el Navigation Safe Args: Es opcional, pero nosotros vamos a utilizarlo. **Safe Args** es una extensión del Navigation Component que permite pasar datos entre Fragments de forma segura y tipada, evitando errores en tiempo de ejecución. La verdad, es que es mucho más sencillo que usar el Bundle().

Abrimos el fichero **build.gradle.kts** (Project) y añadimos entre plugins {...}

```
id("androidx.navigation.safeargs.kotlin") version "2.9.7" apply false
```

No te olvides de actualizarlo de la última versión disponible, si has actualizado antes las librerías.

A continuación, en **build.gradle.kts** (module) añade entre plugins {...}

```
id("androidx.navigation.safeargs.kotlin")
```

Sincroniza de nuevo. Ya estamos listos para empezar a construir la App.

Paso 4 – Fragments: Fíjate que el IDE nos ha creado por defecto un montón de archivos:

- Dos clases con dos Fragments por defecto, con sus respectivos layouts en **res/layouts**.
- Un fichero **content_main.xml** en **res/layouts**.
- Un fichero **nav_graph.xml** en **res/navigation**.

Vamos a adaptar todo esto para imitar el ejemplo del correo.

Paso 5 – Adaptamos: Lo primero es renombrar los Fragments a **EmailDetailFragment** y **EmailListFragment**. Después, renombra sus layouts correspondientes a **fragment_email_detail.xml** y **fragment_email_list.xml**.

Por el momento déjalos sin tocar.

Verás que se generan errores en el **nav_graph.xml**. No te preocupes. Todo está controlado.

Paso 6 – Los Fragments: Vamos a ir construyendo los layouts de los Fragments. No hay que hacer muchos cambios respecto al ejemplo base.

fragment_email_list.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".EmailListFragment">

    <ListView
        android:id="@+id/listViewEmails"
        android:layout_width="match_parent"
        android:layout_height="match_parent" />

</FrameLayout>
```

fragment_email_detail.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:padding="16dp"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:id="@+id/textViewEmail"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:textSize="24sp"
        android:text="Email detalle" />

</LinearLayout>
```

Paso 7 – El Activity_main.xml: Si bien el layout de nuestra actividad está definida en el **Activity_main.xml**, si editas el archivo verás que tiene mucho contenido.

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.coordinatorlayout.widget.CoordinatorLayout xmlns:android="http://schemas.android.com/apk/res-auto"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:fitsSystemWindows="true"
    tools:context=".MainActivity">

    <com.google.android.material.appbar.AppBarLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:fitsSystemWindows="true">

        <com.google.android.material.appbar.MaterialToolbar
            android:id="@+id/toolbar"
            android:layout_width="match_parent"
            android:layout_height="?attr/actionBarSize" />

    </com.google.android.material.appbar.AppBarLayout>

    <include layout="@layout/content_main" />

    <com.google.android.material.floatingactionbutton.FloatingActionButton
        android:id="@+id/fab"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="bottom|end"
        android:layout_marginEnd="@dimen/fab_margin"
        android:layout_marginBottom="16dp"
        app:srcCompat="@android:drawable/ic_dialog_email" />

</androidx.coordinatorlayout.widget.CoordinatorLayout>
```

La **AppBarLayout** y la **FloatActionButton** son una barra de controles y un botón flotante que añade por defecto el Android Studio. No los elimines, porque la primera la vamos a usar y el segundo nos servirá para ver cómo queda. Obviamente, cuando diseñemos nuestra App podremos hacer lo que queramos con ellos.

Fíjate en el **include** que hay entre ambas. Es una referencia a **content_main.xml**. Es decir, que este fichero es el que va a definir todo lo que hay entre ambas barras. Es una forma de tener bien separados los componentes de la vista.

En nuestro caso sólo añadimos los Fragments, podemos dejarlo sin tocar:

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res-auto"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    app:layout_behavior="@string/appbar_scrolling_view_behavior">

    <androidx.fragment.app.FragmentContainerView
        android:id="@+id/fragment_container"
        android:name="androidx.navigation.fragment.NavHostFragment"
        android:layout_width="0dp"
        android:layout_height="0dp"
        app:defaultNavHost="true"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:navGraph="@navigation/nav_graph" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

Paso 8 – El nav_graph.xml: Vamos a arreglar el fichero de navegación de los Fragments. Nótese que estamos utilizando **Safe Args**. Este fichero define principalmente cuatro cosas:

- Número de Fragments que hay
- Cómo se identifica cada Fragment (su id), su nombre (su clase) y su layout.
- De qué Fragment a qué Fragment puedo ir.
- Si este Fragment puede recibir parámetros cuando se navega hacia él.

Veamos un ejemplo de un Fragment:

```
<fragment
    android:id="@+id/emailDetailFragment"
    android:name="es.elorrieta.app.jetpackexample.EmailDetailFragment"
    android:label="@string/second_fragment_label"
    tools:layout="@layout/fragment_email_detail">

    <action
        android:id="@+id/action_emailDetailFragment_to_emailListFragment"
        app:destination="@id/emailListFragment" />

    <argument
        android:name="email"
        app:argType="string" />

</fragment>
```

Aquí podemos ver que:

- El Fragment es:
 - o Id = emailDetailFragment
 - o Nombre es: es.elorrieta.app.jetpackexample.EmailDetailFragment
 - o Su layout: fragment_email_detail
- De qué Fragment a qué Fragment puedo ir: esto se denomina Acciones. En nuestro caso, sólo tendremos una para ir al listado, por lo que:
 - o Id = action_emailDetailFragment_to_emailListFragment
 - o Destino: la id del Fragment al que navegamos, emailListFragment
- Este Fragment además está preparado para recibir un parámetro llamado 'email' de tipo String; si bien no es obligatorio.

¿Te parece que estás escribiendo un fichero de configuración o algo por el estilo? Pues es cierto... Más o menos. Cuando la App se compile, se van a **generar una serie de clases adicionales** sin tu intervención, las cuales se utilizarán para realizar la navegación. Todo esto es gracias a las librerías que has importado y al **Navigation Safe Args**. Esto es lo que hace.

No intentes buscar el código de esa clase, porque no existe. Se autogenera ya compilada.

Explicado más claramente:

- Se generará una clase **EmailDetailFragmentDirections**
- Tendrá un método llamado **actionEmailDetailFragmentToEmailListFragment ()** que permitirá navegar al Fragment emailListFragment.
- Cuando queramos navegar a este Fragment (al EmailDetailFragment) podremos pasarle si queremos un String como una variable llamada 'email'.

Nuestro **nav_graph.xml** debe de quedar algo así. Nótese que hay que indicarle también cual es el primer Fragment que tiene que mostrar al inicio.

```
<?xml version="1.0" encoding="utf-8"?>
<navigation xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/nav_graph"
    app:startDestination="@id/emailListFragment">

    <fragment
        android:id="@+id/emailListFragment"
        android:name="es.elorrieta.app.jetpackexample.EmailListFragment"
        android:label="@string/first_fragment_label"
        tools:layout="@layout/fragment_email_detail">

        <action
            android:id="@+id/action_emailListFragment_to_emailDetailFragment"
            app:destination="@id/emailDetailFragment" />
    </fragment>
    <fragment
        android:id="@+id/emailDetailFragment"
        android:name="es.elorrieta.app.jetpackexample.EmailDetailFragment"
        android:label="@string/second_fragment_label"
        tools:layout="@layout/fragment_email_detail">

        <action
            android:id="@+id/action_emailDetailFragment_to_emailListFragment"
            app:destination="@id/emailListFragment" />

        <argument
            android:name="email"
            app:argType="string" />
    </fragment>
</navigation>
```

Paso 9 – Completa el código: Completa el código de las clases **EmailDetailFragment** y **EmailListFragment**. Es prácticamente igual al de los ejemplos en los que nos basamos, con un par de salvedades:

- **EmailDetailFragment:** le hemos quitado el botón
- **EmailListFragment:** Hay que modificar el evento.

El evento queda así:

```
listView.setOnItemClickListener { _, _, position, _ ->
    val selectedEmail = emails[position]

    val action =
        EmailListFragmentDirections.actionEmailListFragmentToEmailDetailFragment(
            email = selectedEmail
        )

    findNavController().navigate(action)
}
```

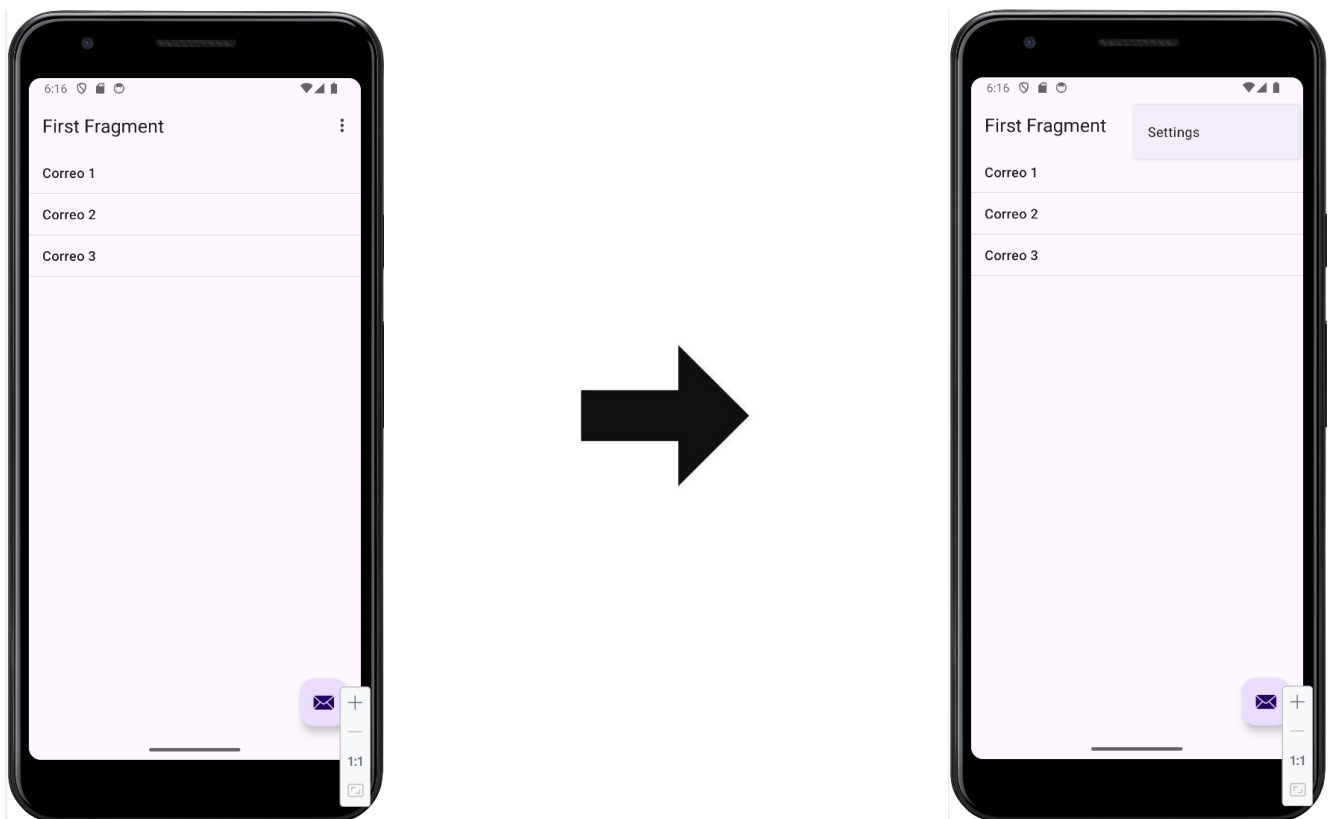
¿De dónde viene la clase EmailListFragmentDirections? ¿Y el parámetro email?

Lo has adivinado. Están definidos en el **nav_graph.xml**.

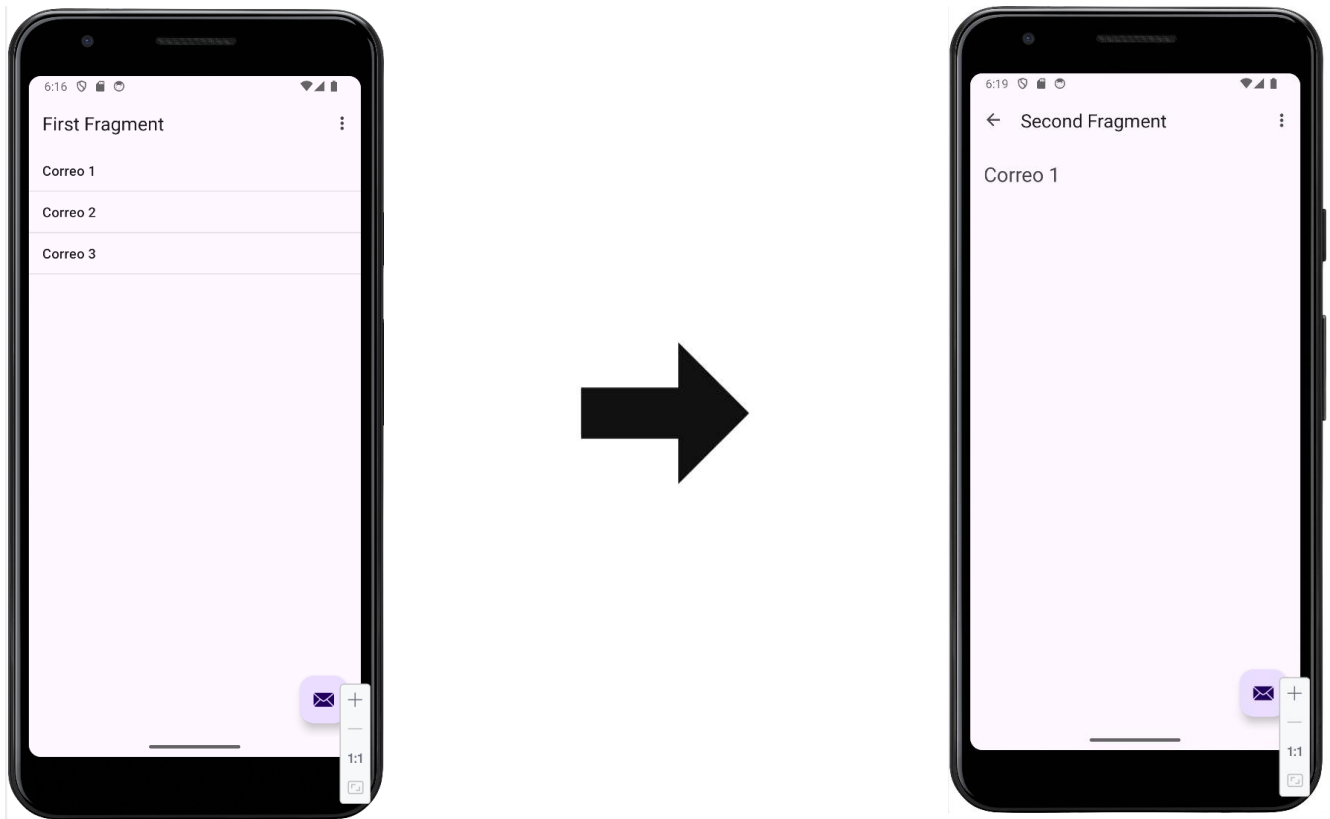
Paso 10 – Finalizando las cosas: Puede que tengas un error en el MainActivity.kt. Posiblemente sea porque todo este código es autogenerado y hemos cambiado el nombre de las cosas. Si tu error es un id a un layout o algo similar, sólo tienes que ponerle el correcto. Por lo demás, no habría que tocar nada.

Verás que se hace referencia a **menu_main**, un fichero en res / menú. Es un recurso que define cómo es la barra de menús superior de la App. Puedes cambiarla como desees. En nuestro ejemplo, simplemente muestra un desplegable con el texto 'Settings'

Puedes ver cómo queda aquí:



La App finalmente funciona de la forma esperada.



Un par de detalles más:

- La flecha en la parte superior izquierda en **EmailDetailFragment** pertenece a la **AppBarLayout**. Si la pulsas, vuelves automáticamente al **EmailListFragment**. Se genera de forma automática.
- El icono de la carta de la parte inferior derecha de ambos Fragments pertenece al **FloatActionButton**. Te recuerdo que decidimos dejar el botón este sin tocar para que veamos cómo nos queda en la App. Obviamente, no hace nada porque no le hemos puesto comportamiento.

Ejercicio 16

Rehaz el ejercicio anterior, pero en este caso, utiliza los conocimientos adquiridos en este documento.